Robust Error Handling in Power Platform

Patterns for Flows, Apps, and Beyond

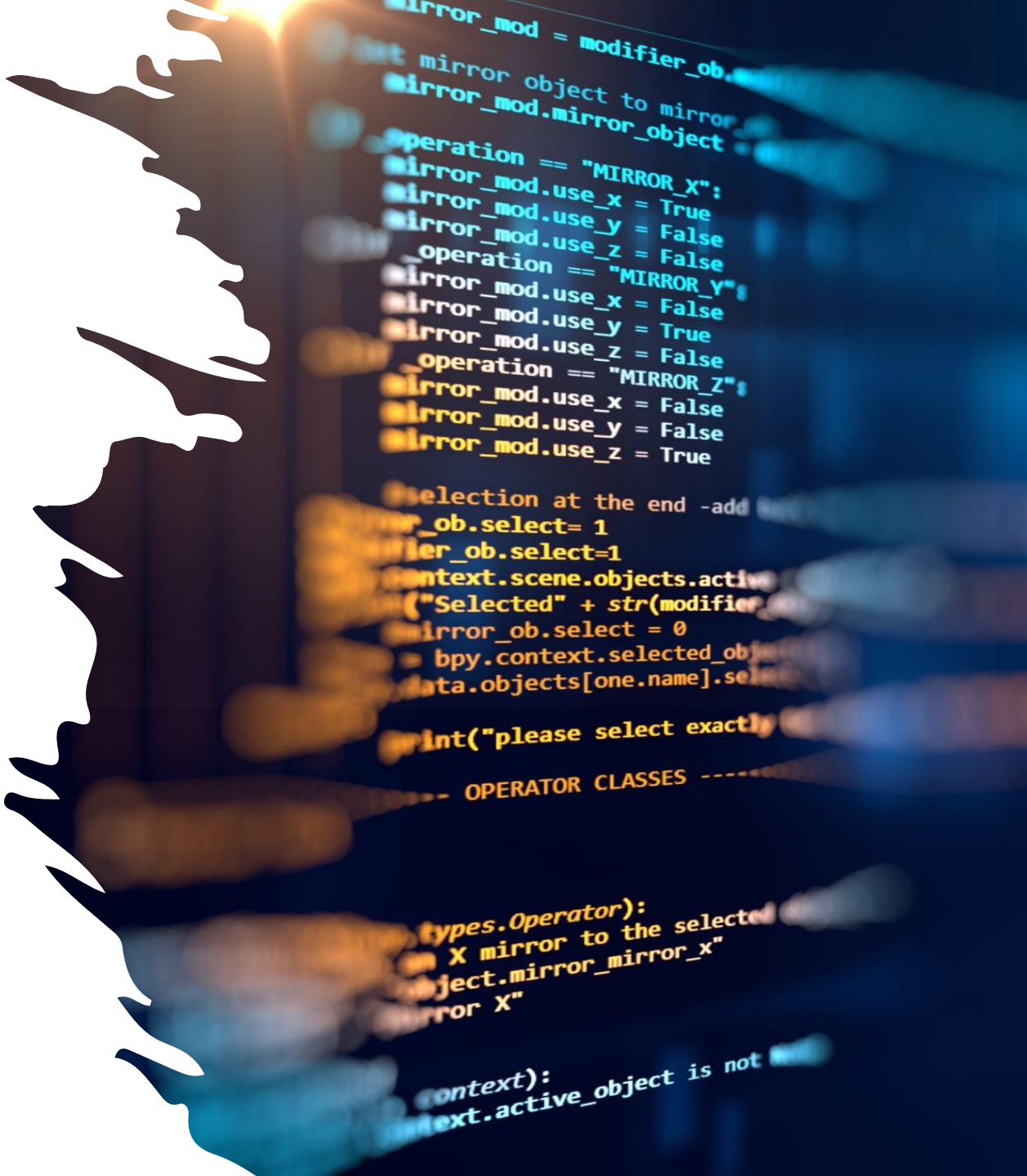
Created By
Divyani Rajani

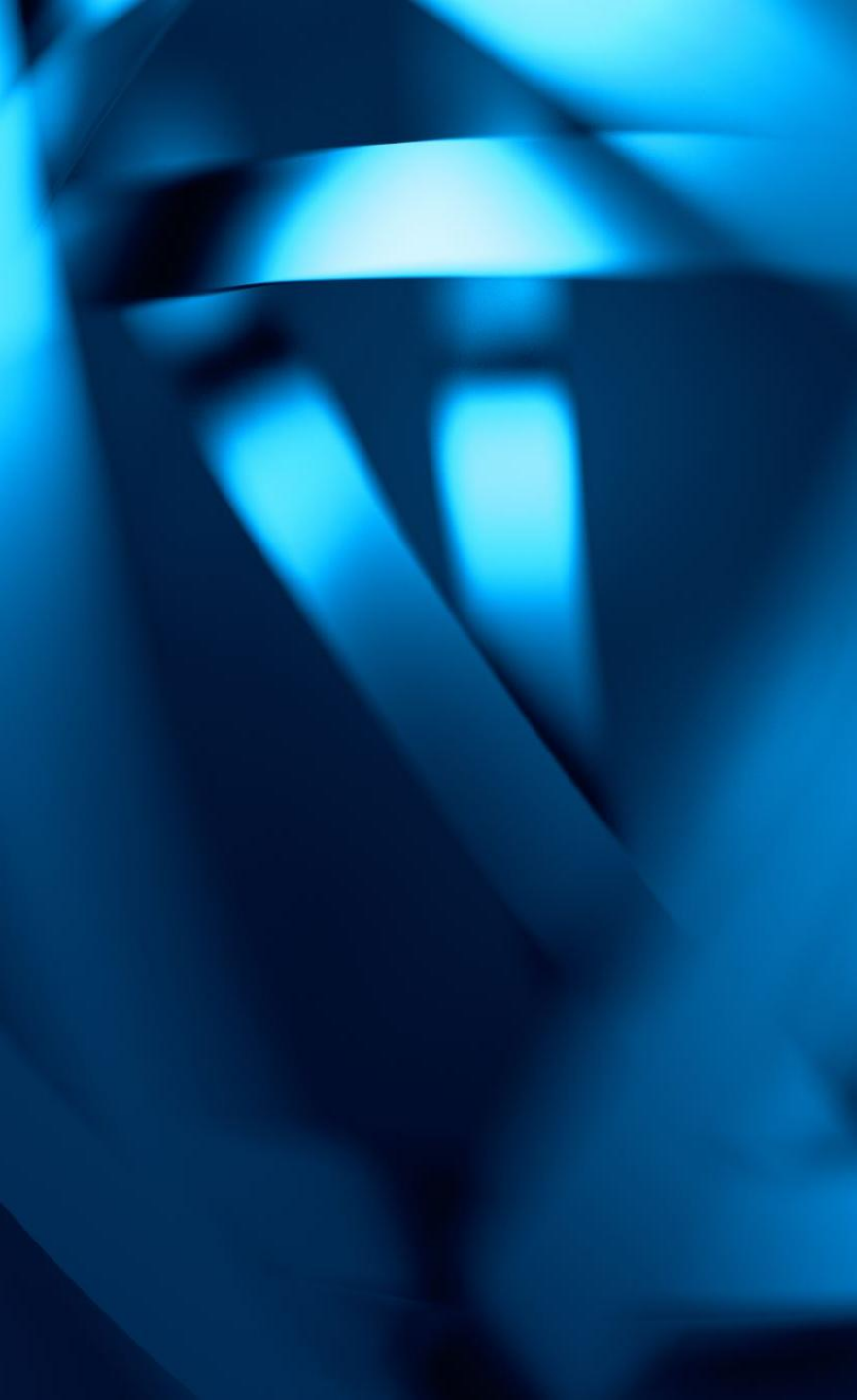
Why Error Handling Matters in Power Platform

- Power Platform : Low code tools like Power Apps, Power Automate, Power BI, Power Pages

- Errors are inevitable – poor handling leads to broken apps and bad user experience

- We need structured patterns for resilience and observability





Common Error Scenarios in Power Platform

- External connector failures (SharePoint, SQL, etc.)
- Invalid or missing user inputs
- API throttling or rate limiting
- Timeout issues or blocked network calls
- Environment misconfiguration or version drift

Error Handling in Power Automate Flows

1

Use **Scope + Run After** to build Try-Catch-Finally logic

2

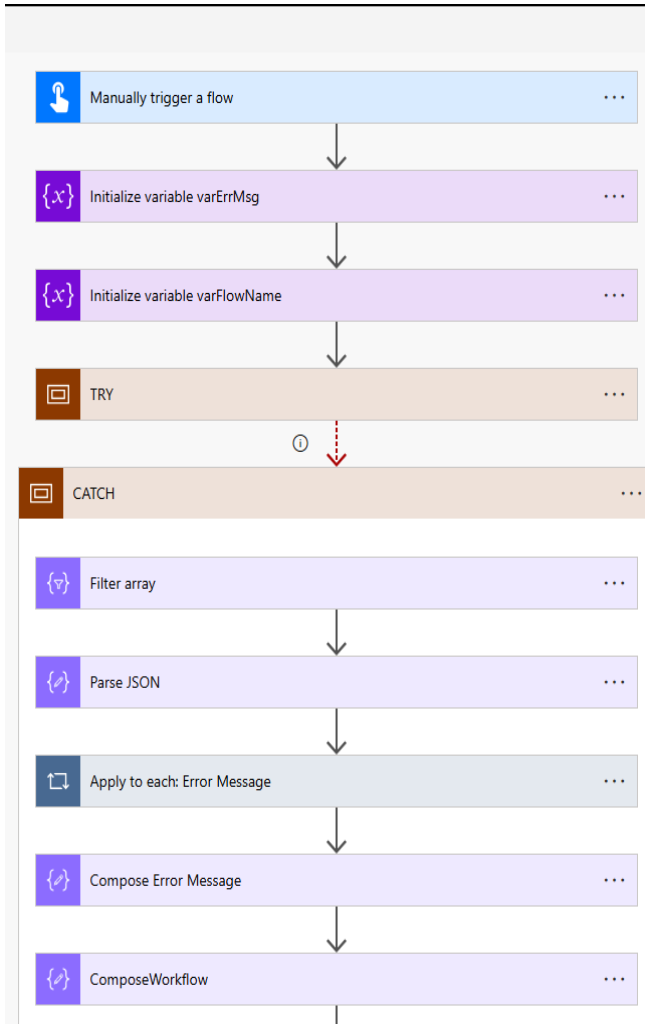
Add **Terminate** action to control flow exit

3

Use **Compose** for dynamic error messages

4

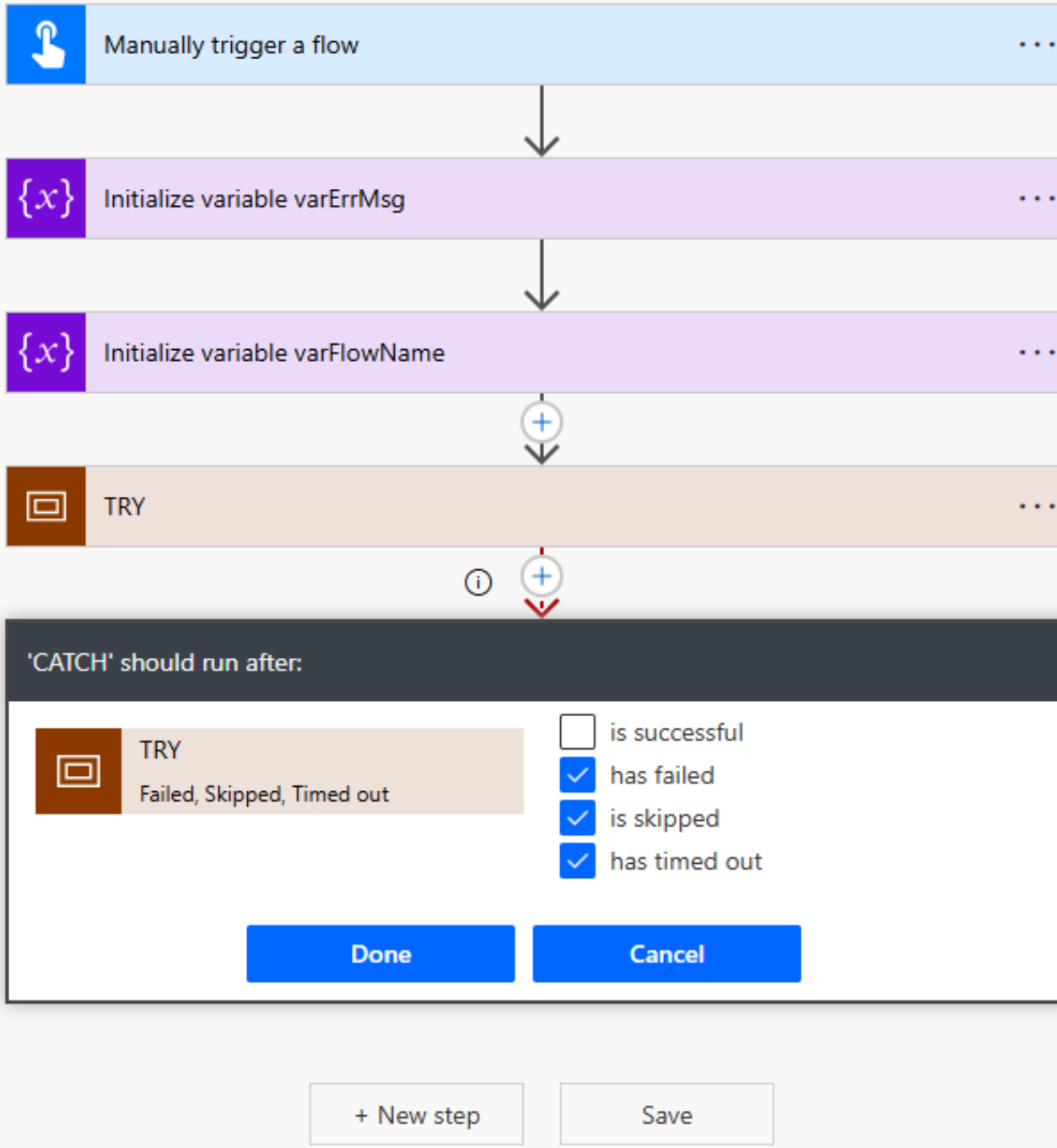
Customize **Retry Policy** on connectors



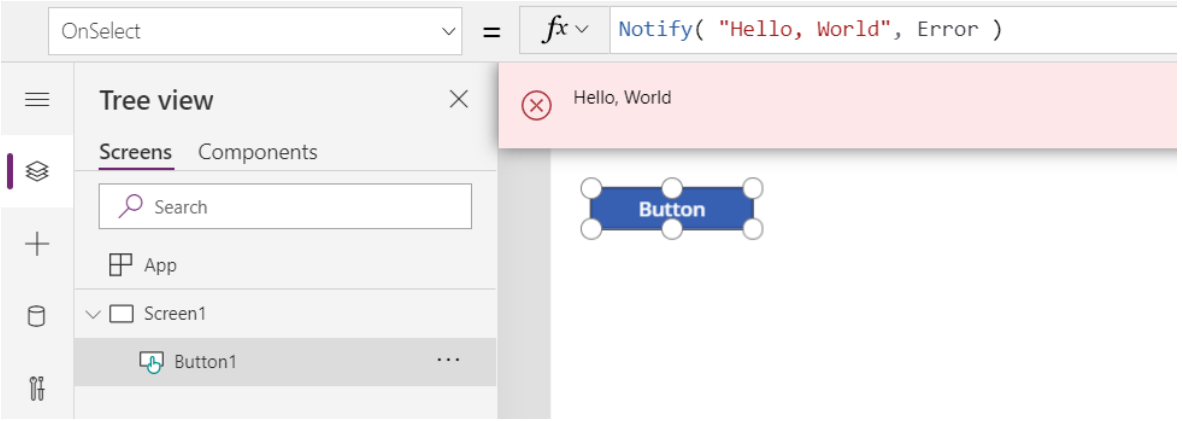
Tip: Use nested scopes to isolate logic and catch errors gracefully

Tools to Improve Flow Reliability

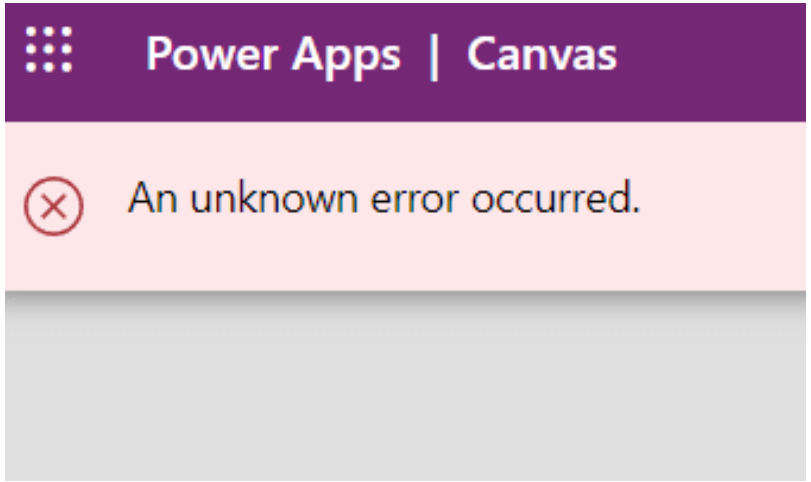
- Track errors with Run History
- Notify users via Teams, Outlook, etc.
- Use Child Flows to encapsulate logic
- Log errors into Dataverse, SharePoint, or Azure



Error Handling in Power Apps



- Use IfError() and Notify() to trap and show messages
- Validate inputs before processing
- Global error state tracking (Set(varError, ...))
- Display friendly error messages instead of system messages
- Pattern: Try-Notify-Retry



Examples..

```
Patch(Orders, ThisItem, {Status: "Approved"});
If(
    !IsEmpty(Errors(Orders)),
    Notify("Approval failed: " & First(Errors(Orders)).Message, NotificationType.Error)
)
```

```
IfError(
    Patch(Orders, Defaults(Orders), {Title: txtOrderTitle.Text}),
    Notify("Something went wrong while saving the order.", NotificationType.Error)
)
```

```
IfError(
    SubmitForm(frmCustomer),
    Notify("Form submission failed. Please try again.", NotificationType.Error)
)
```

```
If(
    IsBlank(txtAmount.Text) || !IsNumeric(txtAmount.Text),
    Notify("Please enter a valid amount.", NotificationType.Warning),
    Set(total, Value(txtAmount.Text) * 1.2)
)
```

```
//capture any unhandled error in the SP List
Patch('TrackerApp Unhandled Errors', Defaults('TrackerApp Unhandled Errors'), {
    AppTitle: "Tracker App",
    Source: FirstError.Observed & "/" & FirstError.Source,
    ErrorMessage: FirstError.Message,
    UserEmail:User().Email
});

//trace in Power Apps Monitor
Trace("Error at " & FirstError.Source & ": " & FirstError.Message);

//Notify user on screen
Error(AllErrors);
```

Error Handling in PowerBI

In Power Query (M Language)

- *try... otherwise to handle risky operations*
  - *E.g. try [Value] / [Quantity] otherwise 0*
- *Remove or replace errors*
  - *Table.ReplaceErrorValues(...)*
  - *Right-click column → "Remove Errors"*

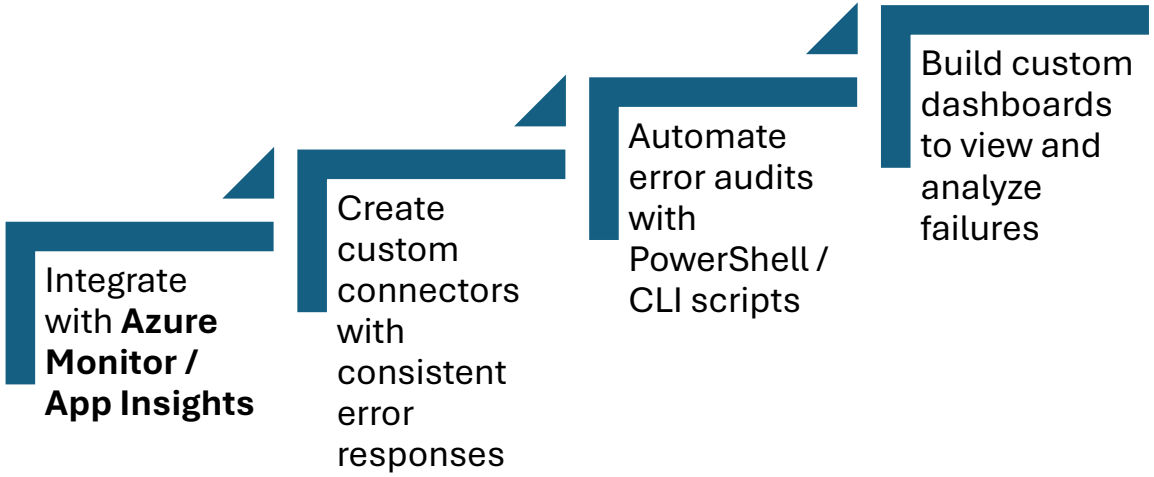
In DAX (Data Analysis Expressions)

- *IFERROR(): Handles calculation*
  - *SafeValue = IFERROR([Num] / [Den], 0)*
- *ISERROR() / ISBLANK(): Conditional logic based on data state*
- *COALESCE(): Substitute nulls with defaults*

Shared Patterns Across Power Platform

- Centralize logging using SharePoint or Application Insights
- Use consistent error schemas across apps and flows
- Configure Environment Variables for logging level control
- Monitor using Power Platform Admin Center

Advanced Error Handling



Best Practices

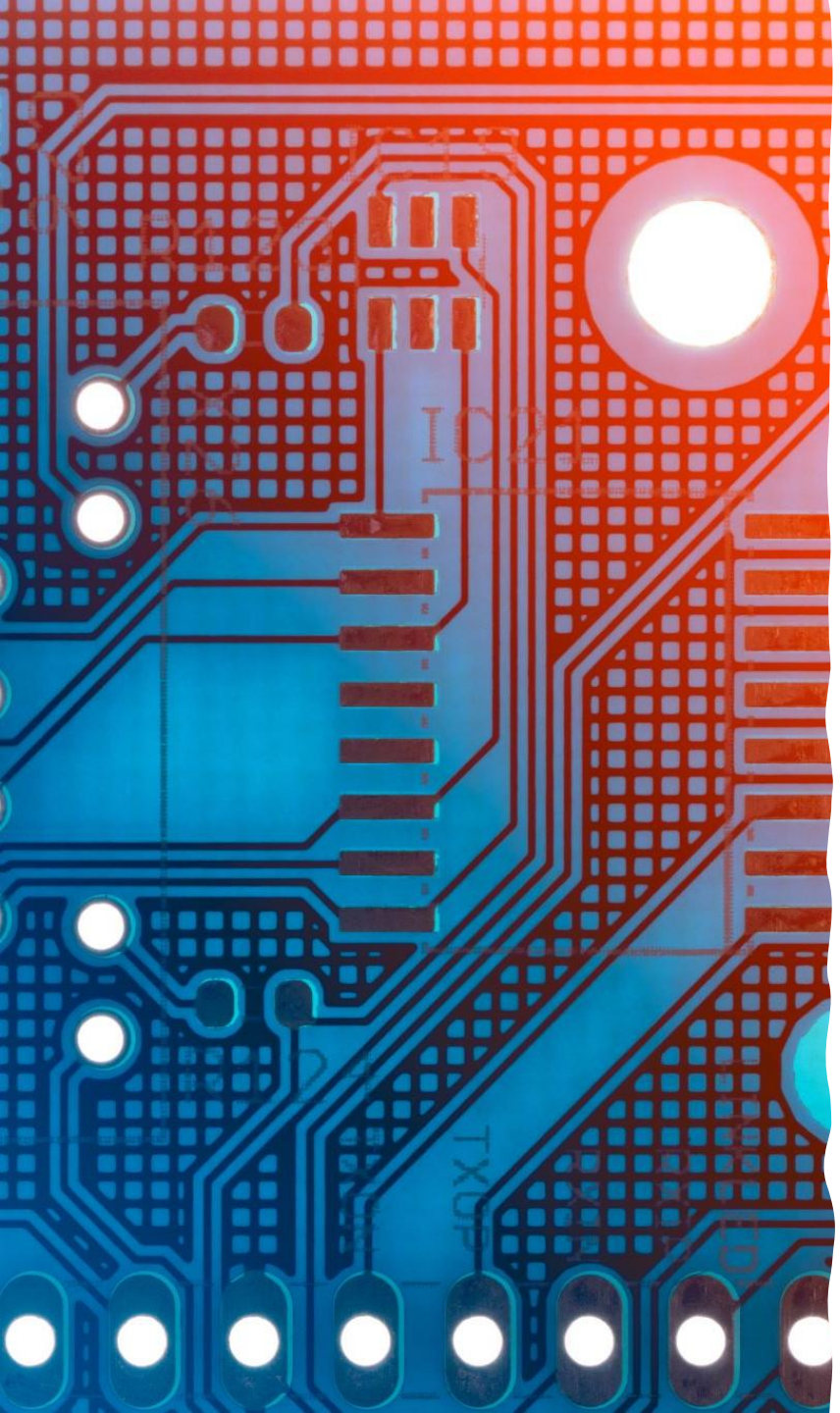
- Use Try-Catch-Finally everywhere
- Avoid exposing technical errors to end users
- Log errors contextually (action, time, user)
- Document expected error paths
- Test error scenarios explicitly before go-live

Summary: Error Handling Capabilities in Power Platform

This table summarizes the capabilities discussed in the presentation and serves as a quick reference guide for developers.

For detailed implementation steps, refer to the respective slides

Area	What Can Be Handled	How It's Handled	Reference Slide
Power Automate (Flows)	Runtime errors, API failures, timeouts, missing data	Configure ***Configure run after, Scope, Try-Catch-Finally, Terminate***, custom notifications	*Error Handling in Power Automate Flows*
Power Apps	API call failures, validation errors, user input issues	Use ***IfError, Notify, Error*** functions, error-aware controls, patch error handling	*Error Handling in Power Apps*
Power BI	Data refresh failures, data source issues, DAX calculation errors	Scheduled refresh alerts, Power BI service monitoring, TRY/ISERROR in DAX, external monitoring tools	*Error Handling in Power BI*
Shared Patterns	Reusable handling logic, error logging, alerting	Common design patterns (e.g., logging table in Dataverse/ SharePoint, unified error message format, etc.)	*Shared Patterns Across Power Platform*
Tools for Flow Reliability	Resiliency in automated flows	Use Retry policies, Concurrency control, Timeout configuration, Scopes	*Tools to Improve Flow Reliability*
Examples (Code Snapshots)	Real-life implementation examples	Code snippets for Power Apps error handling	*Example..*
Advanced Best Practices	Enterprise-level error tracking, observability, governance	Centralized logging, standard templates, monitoring integrations (e.g., Azure Monitor, App Insights)	*Advanced Error Handling - Best Practices*
Why It Matters	Impact of poor error handling on user trust, performance, and reliability	Business justification, user-centric examples to show importance of good error handling	*Why Error Handling Matters*



Resources & References

- Power Automate Error Handling
- Power Apps Error Functions
- Power Platform Community and GitHub
- Microsoft Learn modules on Power Platform

Thank You!